

1 True or False

1. Hosts usually perform the iterative DNS resolution process themselves.

Solution: False; hosts use local DNS servers to perform DNS lookup.

2. Every zone always has at least 2 name servers.

Solution: True.

3. When looking up a root server, BGP will use unicast to find the correct root server.

Solution: False; BGP uses anycast to find the closest root server.

4. A client can establish a TCP connection with a root server.

Solution: False; TCP requires keeping the state, but anycast does not guarantee all the packets of the same connection are sent to the same root server.

5. Most queries to DNS root servers are for nonexistent TLDs.

Solution: True.

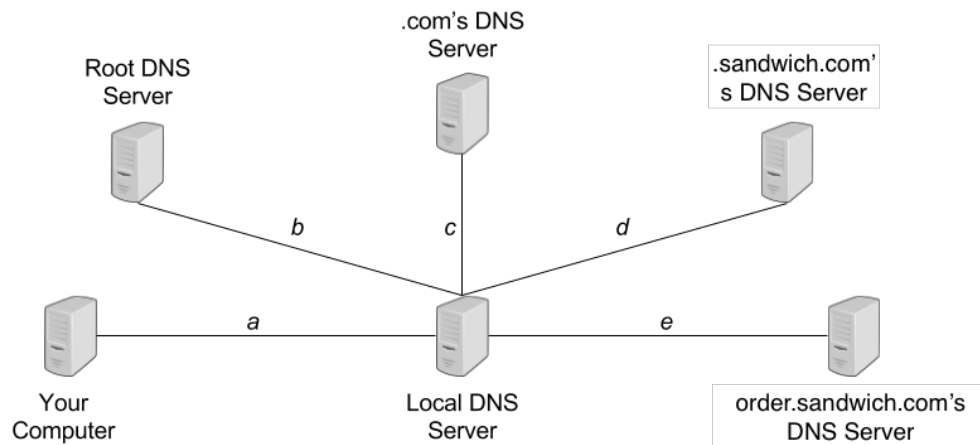
2 DNS Record Types

Write the corresponding DNS record type on the right column.

Information	Record Type
Name to IPV4 Address Mapping	A
Name to IPV6 Address Mapping	AAAA
Name Server	NS
Human Readable Information (Often Used for Site Verification)	TXT
General Name-to-Service Mapping	SRV
Mail Exchanger	MX
Canonical Name	CNAME

3 DNS

A sandwich ordering website `www.order.sandwich.com` is accepting online orders for the next T minutes. Consider the following setup of DNS servers, with annotated latencies between servers:



Assume that the latency between your computer and the website's server is t , that once you send an order for a sandwich you must wait for a confirmation response from the website before issuing another, and that your computer does not cache website's IP address.

Meta: If students ask whether these DNS servers use iterative or recursive DNS lookups, ask which would be more performant for major DNS servers with heavy traffic. (This was covered in lecture.)

In each of the following three scenarios below, determine how many sandwiches you can order for the next T minutes:

- (1) Your local DNS server doesn't cache any information.

Solution: Your computer begins by issuing a DNS query for `www.order.sandwich.com` to its local DNS server, which takes time a . Your local DNS server then iteratively queries the root DNS server, `.com's` DNS server, `.sandwich.com's` DNS server, and `order.sandwich.com's` DNS server, which takes time $2b + 2c + 2d + 2e$. It then returns the result of the query to you, which takes time a . Your computer can then issue a sandwich request to `www.order.sandwich.com`, which responds with an order confirmation, which takes time $2t$. The total time per query is hence

$$2a + 2b + 2c + 2d + 2e + 2t$$

Each sandwich order takes this much time, since our local DNS server doesn't cache the IP address corresponding to `www.order.sandwich.com`, and so the number of orders we can make in this time is

$$\# \text{ sandwiches} = \left\lfloor \frac{T}{2a + 2b + 2c + 2d + 2e + 2t} \right\rfloor$$

(rounding down to the nearest integer with the floor function $\lfloor x \rfloor$).

- (2) Your local DNS server caches responses, with a time-to-live $L \geq T$.

Solution: If $L \geq T$, then this effectively means that once the result of the DNS query for `www.order.sandwich.com` is cached, it will remain cached in our local DNS server until the website's

server ultimately goes down. The first query thus takes the same amount of time as a query from the previous part

$$2a + 2b + 2c + 2d + 2e + 2t$$

so long as this time is less than T . All subsequent queries only take time $2a + 2t$ thanks to caching. Hence we can model the number of sandwiches we get as follows:

$$\# \text{ sandwiches} = \begin{cases} 1 + \left\lfloor \frac{T - (2a + 2b + 2c + 2d + 2e + 2t)}{2a + 2t} \right\rfloor & T \geq 2a + 2b + 2c + 2d + 2e + 2t \\ 0 & T < 2a + 2b + 2c + 2d + 2e + 2t \end{cases}$$

- (3) Let $T = 600$ seconds and $a = b = c = d = e = t = 1$ second. Your local DNS server caches responses with a finite time-to-live of 30 seconds.

Solution: When you make your first DNS query, the response gets cached in your local DNS server at time $a + 2b + 2c + 2d + 2e = 9$ seconds, and will remain cached for the next 30 seconds, until time 39.

Once the response is cached, your local DNS server sends it to your computer (which arrives at time 10) and then you issue a sandwich request (and receive confirmation at time 12). This gives you $39 - 12 = 27$ additional seconds to make more requests until the TTL for the cached entry expires, during which time you can make $\lceil \frac{27}{4} \rceil = 7$ additional sandwich requests, each request is completed at time 12, 16, 20, 24, 28, 32, 36 and 40.

At this point, the cached DNS response has expired, and 40 seconds have elapsed. The events above can repeat a maximum of $\frac{600}{40} = 15$ times in our 600 second window of opportunity, and so we can order at most $15 \cdot (7 + 1) = \boxed{120 \text{ sandwiches}}$. Not bad!